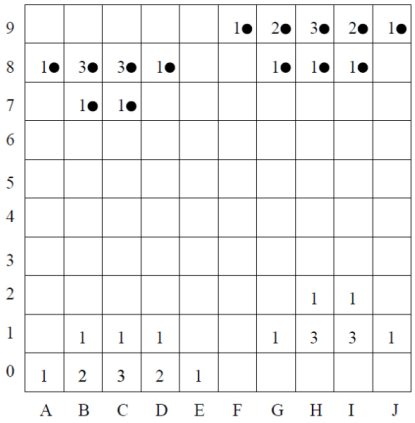
AI : Report

TD1-WARGAME

Written by : Waneeza Hira AWAN

Group : 1

M1 SIC, 2026-2027.



1. Game Mechanics and Successor Generation

To begin this assignment, I first had to understand how this simplified wargame works. The game is played on a 10x10 board with two players. The objective is simple: either capture all enemy pieces, or reach the opponent's back rank with at least one piece to win right away. Each turn, a player can move only one piece by one square in any of the 8 directions. When attacking an enemy piece, combat is resolved by comparing the total strength of surrounding friendly pieces versus enemy pieces across the 8 neighboring squares. Before coding the decision-making logic, my priority was to give the computer a way to find all possible moves for any board state.

In the template code, there was no function to list valid moves. Since an AI needs to test every option during its turn, I created a simple Coup struct containing (l1, c1) for the starting square and (l2, c2) for the destination. Storing only these coordinates is lightweight and avoids duplicating entire 100-cell boards needlessly. To collect all available actions, I used a singly linked list (ListeCoups). Because the number of possible moves changes constantly during a match, a linked list lets me allocate just what is needed. I also added ajouter_coup to insert new moves at the front and liberer_coups to free the allocated memory after checking the moves.

The generation itself happens in generer_coups. It loops over the whole board to find pieces belonging to the current player. For every piece found, it checks the 8 neighboring squares using relative offsets. Instead of rewriting all the movement rules, I called f_test_mouvement, which was already provided. This function handles boundary checks and prevents moving onto an ally's square. If the move is valid, it gets added to the list. With this done, the AI has its candidate moves ready for the Minimax search tree.

For interconnection with other AIs, here is the list of elements I added to the provided template: the structures Coup and ListeCoups; the functions ajouter_coup, liberer_coups, generer_coups, f_max_ab, f_min_ab and test_victoire; and the global variables algo_ia (1 = Minimax, 2 = Alpha-Beta) and nb_noeuds_visites (node counter). I modified f_min, f_max, f_IA, f_eval and the main function (algorithm selection menu). The search depth is fixed to 3, as required for the tournament.

2. Implementing the Minimax Decision Logic

After having a move generator, my next task was to code the core decision-making loop using the Minimax concept. Since this is a turn-based, two-player game, the idea is fairly simple: my AI wants to pick the move that leads to the highest possible score, while keeping in mind that the opponent will always try to pick the move that hurts my position the most. To make the code readable and directly follow what we saw in class, I decided to split this logic into two mutually recursive functions: f_max for my turns and f_min for the opponent's turns.

When writing these functions, my main concern was how to handle the board copies without slowing down the program or causing memory leaks. At first, allocating dynamic memory with malloc or f_raz_plateau inside every branch seemed like an easy way out, but it quickly became messy and inefficient with thousands of calls. Instead, I chose to allocate a local board array directly

on the stack (Pion copie[...]) inside the loop. For every legal move, I duplicate the current state using f_copie_plateau, execute the action with f_bouge_piece, and recursively call the opposite function with profondeur - 1 and -joueur to flip the turn.

Crucially, before expanding any node, I added a fast terminal condition using a dedicated helper, test_victoire. , which detects a breakthrough of the back rank. If test_victoire(plateau) != 0 or once we hit profondeur <= 0, the state is evaluated from my AI's point of view via f_eval. Checking test_victoire prevents the algorithm from exploring branches beyond a breakthrough that has already ended the match, without it, the simulation could "capture" the intruding piece in a later ply and wrongly believe the situation was safe. The elimination of a whole army is not detected by test_victoire, but it is handled by f_eval at the leaves. Finally, I make sure to call liberer_coups at the end of each function to free the linked list before returning the best score.

3. Completing the Decision Pipeline and Positional Evaluation

Once the mutual recursion of f_min and f_max was in place, I connected the search engine to the actual match through f_IA. This function acts as the root of the Minimax tree. It generates all legal moves for the active player, simulates each candidate action on a local board buffer, and calls f_min at depth - 1 to anticipate the opponent's counterplay. Unlike the internal recursive nodes that only propagate scalar values, f_IA stores the origin and destination coordinates of the move yielding the highest score and executes it directly on plateauDeJeu.

Simultaneously, I designed a complete heuristic in f_eval. Moving away from a raw material count was essential to prevent the engine from idling when no immediate captures were available. The function first detects terminal states by itself (a piece on the opponent's back rank, or an army reduced to zero pieces) and assigns +5000 or −5000. Otherwise, it weighs the material balance using f_valeur (×20) and f_nbPions (×10), adds an advancement bonus (×2) proportional to how far each piece has pushed forward, and subtracts a safety penalty for enemy pieces close to our back rank (see section 5).

4. Pruning the Search Tree with Alpha-Beta

While plain Minimax gives the correct optimal move, exploring every branch becomes impractical as the game tree expands. To make the search much faster without changing the final decision, I implemented Alpha-Beta pruning. Instead of overwriting my existing f_min and f_max implementations, I chose to write two new functions, f_min_ab and f_max_ab. This keeps the baseline implementation intact for direct comparison and code clarity.

In these functions, I introduced two bounding parameters: alpha, representing the minimum score the maximizing player is assured of, and beta, representing the maximum score the minimizing player will allow. In f_max_ab, whenever a candidate move raises valeur_max above alpha, alpha is updated, if it matches or exceeds beta, a cutoff occurs (break) because the opponent at the parent node already has a better alternative. Symmetrically, f_min_ab adjusts beta downward and breaks as soon as beta <= alpha. At the root in f_IA, the search starts with the widest window [-infinity, +infinity]. I also added a global counter, nb_noeuds_visites, reset before every ply, to record the exact volume of nodes explored per move. This gives me concrete experimental metrics to verify the efficiency of the pruning.

5. Enhancing the Heuristic with Depth Weighting and Baseline Defense

While connecting the search tree was an important milestone, testing previous iterations highlighted significant tactical flaws in the evaluation function. The most glaring issue was that terminal states returned a flat score of +5000, making the engine completely indifferent between winning immediately or delaying the winning blow across future turns.

To fix this, I redesigned f_eval to accept the remaining search depth as an additional parameter. Terminal wins now return 5000 + profondeur, while terminal losses return -5000 - profondeur. This simple mathematical adjustment introduces time preference: an immediate breakthrough yields a strictly higher payoff than a delayed one, actively punishing stagnation. In addition, I introduced a defensive safety term called danger_fond. In earlier tests, opposing pieces could sneak into our back lines unnoticed because the material balance remained unchanged. By applying a progressive penalty whenever an enemy unit penetrates within two squares of our goal line, the AI is now forced to react defensively and contest perimeter breaches. Finally, I increased the coefficient on piece differential to encourage favorable combat exchanges. These adjustments transform the engine from a passive search loop into a goal-driven, aggressive tactical player.

6. Interactive Algorithm Selection and Decisive Victory Termination

To provide rigorous empirical benchmarks for the project report, I integrated an interactive menu at program launch allowing the user to toggle between plain Minimax and Alpha-Beta pruning. Both algorithm variants increment the same shared exploration counter (nb_noeuds_visites), making it straightforward to record exact side-by-side performance metrics across identical board configurations.

Running full games with this setup exposed a critical tactical gap: while the heuristic awarded high scores to advanced pawns, the search tree at depth 3 still suffered from horizon ambiguity when picking between finishing the game and playing alternate safe moves. To make the endgame completely decisive, I leveraged test_victoire directly inside the root loop of f_IA. Before querying child branches, the AI checks if test_victoire(copie) == joueur. If a candidate action immediately invades the enemy back rank, it is instantly awarded an overriding score of 10000. As remarked in the execution logs of a full AI vs AI game, the final move of the game is J8 -> J9, which is selected with the overriding score of 10000 as soon as the winning breakthrough is available at the root. This check forces the engine to seal the victory on the spot rather than idling in place. The use of test_victoire at both root and recursive levels ensures clean game convergence and provides stable benchmarks.

7. Empirical Search Space Benchmarks and Comparative Analysis

To verify the theoretical advantage of Alpha-Beta pruning over standard Minimax, I ran a complete AI vs AI game (88 plies) once with each algorithm, at search depth profondeur = 3, and logged the number of explored nodes for every move with the shared counter nb_noeuds_visites. In my implementation, Black (x) plays first, so odd plies are played by Black and even plies by White (o). Both algorithms chose exactly the same move at all 88 plies, and the game ended with a White victory (J8 -> J9). This confirms that the cutoffs do not change the final decision.

Game Phase	Played Move	Plain Minimax Nodes	Alpha-Beta Nodes	Pruning Efficiency
Opening Move	Ply 1: J9 → J8	81,732 nodes	2,069 nodes	97.44% **pruned**
Opening	Ply 2: I2 → I3	85,622 nodes	3,268 nodes	96.18% **pruned**

Midgame Advance	Ply 7: J8 → J7	142,276 nodes	4,232 nodes	97.03% **pruned**
Combat	Ply 13: I8 → H7	183,531 nodes	18,434 nodes	89.96% **pruned**
Combat Peak	Ply 23: I8 → J7	209,532 nodes	22,910 nodes	89.07% **pruned**
Decisive Breakthrough	Ply 88: J8 → J9	117,630 nodes	138 nodes	99.88% **pruned**

Over the 88 plies, the pruning efficiency ranges from 68.4% to 99.9%, with an average of 92.2%. Plain Minimax explores between roughly 80,000 and 210,000 nodes per move, whereas Alpha-Beta usually needs only a few thousand. The gain is smaller in tactical positions (around 90% at plies 13 and 23), where many moves have close scores and fewer branches can be discarded early. It is almost total on the last move (99.9%) because the root check of test_victoire returns the winning score immediately, so the search tree is hardly explored. The gain could be improved further by ordering the moves (for example, trying captures first), since the list built by ajouter_coup is currently explored in reverse scan order.

8. Critical Reflection and Project Conclusion

Working through these steps revealed critical insights into adversarial game tree search. First, a search that does not stop on terminal states gives wrong evaluations: before adding test_victoire in f_max, f_min and their Alpha-Beta versions, the simulation could continue after a breakthrough and misjudge the threat. Second, the depth-weighted terminal score (5000 + profondeur) was necessary to make the AI choose the fastest win and the slowest defeat instead of being indifferent between them.

In conclusion, this project demonstrated that Alpha-Beta pruning reliably cuts the search space while keeping exactly the same decisions as Minimax: in a full 88-ply game, it removed 92.2% of the nodes on average (between 68.4% and 99.9%). However, strategic play depends entirely on the heuristic. Combining material balance, progressive advancement, a penalty for enemy infiltration near our back rank, depth-weighted terminal rewards and an explicit instant-win check with test_victoire allowed our AI agent to play decisively without looping or hesitating.